



Faculty of Computing

Department of Computing

Laboratory Manual

CS-250 : Data Structure and Algorithms

Class: BESE 16AB • Semester: Fall 2026 – 3rd Semester

Experiment No. 01

Pointers in C++

Date	11 September 2026
Time	09:00 AM – 11:50 AM & 2:00 PM – 4:50 PM
Instructor(s)	Dr Faisal Shafait
Lab Engineer	Hamza Fayyaz
Mapped CLO	CLO2 : Solve a given real time problem by applying appropriate data structure and algorithm. CLO3 : Practice programs using the latest IDEs ensuring testing, documentation and packaging of programs as per standards practices applicable to the software industry.:

1 Introduction

This lab is about pointers. In C++, a pointer refers to a variable that holds the address of another variable. Like regular variables, pointers have a data type. For example, a pointer of type integer can hold the address of a variable of type integer. A pointer of character type can hold the address of a variable of character type.

2 Objectives

This lab will revise the old concepts taught to the students in the previous semesters.

3 Tools/Software Requirement

Visual Studio C++

4 Description

Pointers are used to point towards a particular memory address. In this lab we will use the pointers and perform task with the help of them.

5 Practice Tasks

5.1 Problem 1: Pointer Basics

Problem: Write a C++ program that declares an integer variable `x` and a pointer variable `ptr`. Assign the address of `x` to `ptr`, modify the value of `x` through the pointer, and print the new value of `x`.

Solution:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int x = 10;
6      int *ptr = &x; // Assign address of x to ptr
7
8      cout << "Original value of x: " << x << endl;
9
10     // Modify value of x using the pointer
11     *ptr = 20;
12
13     cout << "New value of x after modifying through pointer: " << x << endl;
14
15     return 0;
16 }
```

Explanation:

- `ptr` is a pointer that stores the address of the variable `x`.
- `*ptr` is used to modify the value of `x` via the pointer.

Output:

```
1  Original value of x: 10
2  New value of x after modifying through pointer: 20
```

5.2 Problem 2: Array and Pointer Relationship

Program:

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int arr[5] = {1, 2, 3, 4, 5};
6     int *ptr = arr; // Pointer to the array
7
8     cout << "Array values: ";
9
10    for (int i = 0; i < 5; ++i) {
11        cout << *(ptr + i) << " "; // Access array using pointer
12    }
13
14    cout << endl;
15
16    return 0;
17 }
```

Output:

```
1 Array values: 1 2 3 4 5
```

Explanation: The pointer `ptr` points to the first element of the array. The array elements are accessed using `*(ptr + i)` which dereferences the pointer and offsets the value by `i`.

5.3 Problem 3: Swap Two Numbers Using Pointers

Program:

```
1 #include <iostream>
2 using namespace std;
3
4 // Function to swap two numbers using pointers
5 void swap(int *a, int *b) {
6     int temp = *a;
7     *a = *b;
8     *b = temp;
9 }
10
11 int main() {
12     int num1 = 10, num2 = 20;
13
14     cout << "Before swapping: num1 = " << num1 << ", num2 = " << num2 << endl;
15
16     // Swap using pointers
17     swap(&num1, &num2);
18
19     cout << "After swapping: num1 = " << num1 << ", num2 = " << num2 << endl;
20
21     return 0;
22 }
```

Explanation: The `swap` function takes two integer pointers as arguments and swaps the values by dereferencing the pointers. The addresses of `num1` and `num2` are passed to the `swap` function.

Output:

```
1 Before swapping: num1 = 10, num2 = 20
2 After swapping: num1 = 20, num2 = 10
```

5.4 Problem 4: Dynamic Memory Allocation Using Pointers

Program:

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // Dynamically allocate memory for an array of 5 integers
6     int *arr = new int[5];
7
8     // Assign values to the array
9     for (int i = 0; i < 5; ++i) {
10         arr[i] = i + 1;
11     }
12
13     // Print the array values
14     cout << "Array values: ";
15     for (int i = 0; i < 5; ++i) {
16         cout << arr[i] << " ";
17     }
18     cout << endl;
19
20     // Deallocate the memory
21     delete[] arr;
22
23     return 0;
24 }
```

Explanation: The new operator is used to allocate memory dynamically for an array of integers. After using the array, the delete[] operator is used to deallocate the memory to avoid memory leaks.

Output:

```
1 Array values: 1 2 3 4 5
```

5.5 Problem 5: Using Void Pointers and Typecasting

Program:

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int a = 42;
6     double b = 3.14;
7     char c = 'Z';
8     void *ptr; // Generic pointer
9
10    // Point to integer and typecast
11    ptr = &a;
12    cout << "Integer value: " << *(int*)ptr << endl;
13
14    // Point to double and typecast
```

```

15     ptr = &b;
16     cout << "Double value: " << *(double*)ptr << endl;
17
18     // Point to char and typecast
19     ptr = &c;
20     cout << "Character value: " << *(char*)ptr << endl;
21
22     return 0;
23 }

```

Explanation: `void*` is a generic pointer type that can hold the address of any data type. To access the value, you must explicitly typecast it back to the correct type (`(int*)`, `(double*)`, `(char*)`). This allows a single pointer to be reused for multiple data types, but the programmer must handle type safety manually.

Output:

```

1 Integer value: 42
2 Double value: 3.14
3 Character value: Z

```

6 Lab Tasks

Task 1.

Write code to find the memory in bytes occupied by `int`, `long`, `double`, `float` and `char`.

Task 2.

Consider the following program and answer the questions.

```

1 void main()
2 {
3     int a, *pa; // Statement 1
4     pa = &a; // Statement 2
5     cout << "pa = &a --> pa = " << pa << endl << endl;
6     pa = pa + 1; // Statement 3
7     cout << "pa = pa + 1 --> pa = " << pa << endl << endl;
8     pa = pa + 3; // Statement 4
9     cout << "pa = pa + 3 --> pa = " << pa << endl << endl;
10    pa = pa - 1; // Statement 5
11    cout << "pa = pa - 1 --> pa = " << pa << endl << endl;
12 }

```

Output:

```

pa = &a --> pa = 0027FF0C
pa = pa + 1 --> pa = 0027FF10
pa = pa + 3 --> pa = 0027FF1C
pa = pa - 1 --> pa = 0027FF18

```

- Why does the memory address stored in pointer “pa” vary by 4?
- Will the address still vary by 4 if the data type of the above mentioned code changed from “int” to “long”? Explain your answer.
- If we try to multiply the address pointed to by “pa” what will happen? Is this logically or programatically correct? Attach screen shot of the output you get when you try this multiplication.

Task 3.

```
1 int list[5] = {3, 6, 9, 12, 15};
2 int *pArr = list;
```

Your task is to write a piece of code that prints all values stored in the array `list` using only pointer variable `pArr`. Do not use the conventional way of printing values by numbering indexes. **Output:**

Task 4.

Write output of the following C++ codes in your document without executing it.

Example code a)

```
1 int x[4] = {0, 4, 6, 9};
2 int *p, a = 3;
3 p = x;
4
5 (*p)++;
6 cout << *p << endl;
7 cout << *(p + 1) << endl;
8 p++;
9 *p = *p + a;
10 cout << *p << endl;
11 p = p + 2; // What is happening here?
12 cout << *p << endl;
```

Output:

Example code b)

```
1 int a, *p, *q;
2 int arr[4] = {0};
3
4 p = arr;
5 q = p;
6
7 *p = 4;
8
9 for (int i = 0; i < 3; i++) {
```

```

10     a = *p;
11     p++;
12     *p = (a + i);
13 }
14
15 for (int j = 0; j < 4; j++) {
16     cout << *q << " ";
17     q++;
18 }

```

Output:

Task 5.

```

1  int a = 5, b = 10;
2  int *pa = &a; // pa and pb are pointer variables of type int.
3  int *pb = &b;
4  int **ppa = &pa; // ppa and ppb are called double pointers or pointers-to-pointers.
5  int **ppb = &pb;

```

- Write code of a function that swaps values of variables a and b. Input to the function should be the address of both the variables.
- Write code of a function that swaps values of the variables a and b using pointer-to-pointer variables ppa and ppb.

Task 6.

Write a C++ program that:

- Declares an integer and a double variable.
- Uses a void* pointer to store the address of each variable one by one.
- Takes input from the user for both variables using typecasting with the void pointer.
- Prints the values back using typecasting.

Program:

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int x;
6      double y;
7      void *ptr;
8
9      // TODO: Point ptr to integer x and take input
10     cout << "Enter an integer: ";
11     // ptr = ...
12     // cin >> *( (??) ptr );
13

```

```
14 // TODO: Point ptr to double y and take input
15 cout << "Enter a double: ";
16 // ptr = ...
17 // cin >> *(??) ptr );
18
19 // TODO: Print the values using typecasting
20 cout << "You entered integer: " << /* typecast ptr to int */ << endl;
21 cout << "You entered double: " << /* typecast ptr to double */ << endl;
22
23 return 0;
24 }
```

7 Deliverables

Compile a single word document by filling in the solution part and submit this Word file on LMS. The name of word document should follow this format, i.e., YourFullName(reg)_Lab#. You must show the implementation of the tasks in a complete Word document to get your work graded. You must also submit this Word document on the LMS.

Note: Students are required to upload the lab on LMS before deadline.

Use proper indentation and comments. Lack of comments and indentation will result in deduction of marks.

8 Lab Rubrics

Software Problem Realization (CLO2)

Assessment	Doesn't meet Expectation (1–3)	Meets Expectation (4–6)	Exceeds Expectation (7)	Marks
Solve a given real time problem by applying appropriate data structure and algorithm. (CLO2)	The Student is unable to understand and outline the problem and doesn't use the relevant method to solve it.	The student requires some guidance to completely comprehend the problem and to differentiate the data structure and algorithm comprehensively.	The student fully understands the given problem, is able to analyze the relevant method to solve it, and develops a detailed program flow.	

Software Tool Usage (CLO3)

Assessment	Doesn't meet Expectation (1)	Meets Expectation (2)	Exceeds Expectation (3)	Marks
Practice programs using the latest IDEs ensuring testing, documenting, and packaging of programs as per standards practices applicable to the software industry. (CLO3)	The student has no idea on how to use the basic tools of the software. The codes have syntax errors, and parts of the codes are missing. Also, they are unable to imitate the required output.	The student has a limited command on the basic tools of the software and operated it under supervision. The codes are correct in terms of their syntax, however, the program output is not always correct.	The student has full command on various tools available in the software. Furthermore, his/her coding is complete and functional, and the program output is correct. Moreover, they can easily manipulate the code to design a particular solution.	

Total Marks: _____